Vivekanand Education Society's Institute Of Technology (VESIT)
Department of Information Technology
Course: Blockchain

Mini Project

Cookbook / Operations Manual

Blockchain Based Anonymous Voting System

Zero-Knowledge Voting Using Groth16 zk-SNARKs, a 2-of-3 EIP-712 Validator Quorum, an Incremental Merkle Tree, and a Gas-Sponsored Anonymous Relayer on a Local EVM

Roll No.	Name
05	Atharva Chaudhari
10	Sameer Jagiasi
28	Prakash Thapa

Group No.: 2

Date: October 26, 2026

Faculty In-Charge: Kajal Joseph

Repository: `https://github.com/AtharvadChaudhari05/AnonymousVotingSystem`

1 Overview

The Anonymous Voting System is an EVM-based electronic voting platform that keeps every vote private while still proving that every vote is valid. It combines Groth16 zero-knowledge SNARKs, a MiMC-Sponge incremental Merkle tree of depth 20, a 2-of-3 EIP-712 validator signature threshold, a no-admin smart contract with automatic phase transitions, and an anonymous gas-sponsoring relayer with a FIFO nonce queue. It was built to overcome the ten limitations (L1–L10) listed in the IEEE BCCA 2022 paper "Sancus: an Anonymous and Trustworthy Blockchain-based Electronic Voting Architecture".

> **Glossary — "commitment" and "nullifier":** a *commitment* is the public hash MiMCSponge(nullifier, secret) stored as a leaf in the Merkle tree to show that a voter is registered. A *nullifier* is a private random number; only its one-way hash (the *nullifierHash*) is revealed when voting, and the contract remembers it so the same voter can never vote twice.

1.1 Privacy and Security Layers

1. **No-Admin Governance** — the contract has no owner and no privileged functions; title, candidates, validators, threshold and timestamps are immutable after deployment.
2. **Automatic Phase Transitions** — the phase is computed from `block.timestamp`: NotStarted, Registration, RegistrationEnded, Voting, Ended.
3. **Multi-Validator Eligibility** — 2 of 3 distinct validators must sign an EIP-712 approval for the voter's commitment before it enters the tree.
4. **Duplicate-Identity Protection** — each approval covers a salted `identityHash`; the contract records used hashes in `usedIdentityHashes`.
5. **Zero-Knowledge Anonymity** — a vote reveals only the nullifierHash and a Groth16 proof of Merkle membership; it cannot be linked to a commitment or identity.
6. **Gas-Sponsored Relayer** — registrations and votes are sent by a relayer wallet that pays gas, so the voter's wallet never appears on-chain.
7. **Zero Transaction Loss** — a FIFO queue with sequential nonce tracking prevents collisions and drops under parallel load.

1.2 Core Stack

Layer	Tooling
Blockchain	Solidity 0.8.20, Hardhat 2.22, ethers v5.7 (local Hardhat EVM, chain ID 31337)
Identity & Eligibility	EIP-712 typed signatures, 2-of-3 validator quorum, `identityHash` registry
Zero-Knowledge	Circom 2.1, snarkjs 0.7 (Groth16), MiMC-Sponge over BN254, Merkle tree depth 20
Verification	On-chain `Verifier.sol` using the AltBn128 pairing precompile (`0x08`)
Relayer	Node.js 18+, Express 4, custom FIFO mutex queue, zero-IP logging
Frontend	React 18, Vite 5, vanilla CSS, WebAssembly prover in the browser
Testing	Hardhat + Mocha/Chai (13 tests), custom benchmark and stress-test scripts

2 Prerequisites

- Node.js 18, 20 or 22 (LTS recommended)
- npm (bundled with Node.js)
- Git
- A browser with WebAssembly support, with the MetaMask extension installed
- MetaMask network for the local node: RPC `http://127.0.0.1:8545`, Chain ID 31337
- No API keys, testnet ETH or cloud accounts are needed; everything runs locally

2.1 Install Dependencies

```
git clone https://github.com/AtharvadChaudhari05/AnonymousVotingSystem.git
cd AnonymousVotingSystem
npm install
cd frontend && npm install && cd ..
```

3 Setup

3.1 Compile the Contracts

```
npm run compile
```

Hardhat compiles with two compiler versions (0.8.20 and 0.8.17) with the optimizer enabled (200 runs).

3.2 Election Configuration

The election is defined in `election.config.json` and read by `scripts/deploy.js`. It is frozen on-chain after deployment.

Field	Meaning
electionTitle	Title shown in the UI (default: University Student Council Presidential Election 2026)
candidates	List of candidate names (minimum 2; IDs start at 1)
validators	Three validator addresses (Hardhat accounts 1, 2, 3)
approvalThreshold	Must be 2 (the contract rejects any other value)
timing	Registration duration, pause before voting (10 s), voting duration (default 86 400 s each)

3.3 Start the Local Blockchain and Deploy

Terminal 1:

```
npx hardhat node
```

Terminal 2:

```
npm run deploy
```

The script deploys the MiMCSponge hasher, `Verifier.sol` and `AnonymousVotingSystem.sol`, then writes the addresses and ABI to `deployed-contracts.json` (also copied for the frontend).

3.4 Relayer Environment Variables

All are optional for the local demo.

Variable	Default
PORT	3001
RPC_URL	`http://127.0.0.1:8545`
RELAYER_PRIVATE_KEY	Hardhat account 4 (public development key; replace for any non-local use)

4 Running the System

4.1 Start the Relayer (Terminal 3)

```
npm run relayer
# runs on http://localhost:3001 with zero IP logging
```

4.2 Start the Frontend (Terminal 4)

```
npm run frontend
# open http://localhost:5173
```

4.3 Production Build Check

```
cd frontend
npm run build
```

4.4 Run Benchmarks and Stress Test

```
npm run simulate       # Sancus L3/L4 benchmark (10, 50, 100, 250 voters)
npm run stress-test    # Sancus L5 flood test (100+ concurrent votes)
```

Results are written to `benchmark-results.json`.

5 Frontend Pages Reference

The React app is a single page with role-based tabs. There is deliberately no admin tab.

Tab / Component	Purpose
Phase banner (`PhaseBanner`)	Shows the current election phase and a "Next Phase (Demo)" button that calls the relayer's `/api/advance-phase`.
Voter (`VoterView`)	Three-step wizard: (1) enter student ID, generate the secret note and request approval; (2) submit registration to validators; (3) pick a candidate, generate the zk-SNARK proof and vote through the relayer.
Validator (`ValidatorView`)	Pending registration queue; each validator signs the EIP-712 approval; at 2 of 2 approvals the request can be submitted on-chain.
Results (`ResultsView`)	Live tally per candidate with bar charts and the declared winner once voting ends.
Audit (`AuditView`)	Public zero-trust audit: votes ≤ registered voters, event counts match state, nullifiers unique.
Performance (`PerformanceView`)	Relayer metrics and benchmark results (gas, latency, throughput).

6 Demo Story (Implemented End-to-End)

1. **Register as a voter.** Open the Voter tab, type a student roll number (for example 0510), leave the salt blank and click *Generate Secret Note & Request Approval*. A backup note containing the private nullifier and secret is downloaded; then submit the request to the validators.
2. **Validator approval.** In the Validator tab, sign the request as Validator 1 and Validator 2. The approvals counter reaches 2 / 2.
3. **Register on-chain.** Click *Submit to Blockchain*. The relayer calls `registerVoter`, and the row shows "On-Chain (Leaf 0)".
4. **Advance to voting.** Click *Next Phase (Demo)* in the banner until the badge shows Voting Active.
5. **Cast an anonymous vote.** In the Voter tab choose a candidate and click *Generate zk-SNARK Proof & Vote*. The proof takes about 3.4 seconds in the browser; the relayer submits `vote` and pays gas.
6. **View results.** Open the Results tab for live tallies; advance to Ended to declare the winner.
7. **Audit.** Open the Audit tab to confirm total votes ≤ registrations and that no duplicate nullifiers exist.

7 Testing

```
npm test
# equivalent to: npx hardhat test
```

Runs the 13-test suite in `test/AnonymousVotingSystem.test.js`, which covers governance rules, phase enforcement, validator multi-signatures and zero-knowledge double-vote rejection. The tests are grouped as follows:

Group	What it checks
1. Deployment & No-Admin	Deployer has no special rights; invalid threshold or duplicate validators are rejected (2 tests)
2. Phase Transitions	Phases advance automatically with timestamps (1 test)
3. Registration	Rejects wrong phase, a single or repeated signer, and non-validators; accepts 2 valid signatures and blocks duplicate identities (4 tests)
4. Anonymous Voting	Rejects votes before or after the window, double voting and unregistered commitments; accepts a valid proof (5 tests)
5. End-to-End Tally	Full election with 3 voters, correct tally and winner (1 test)

8 Smart Contracts

Three Solidity contracts, deployed together by `scripts/deploy.js`.

Contract	Purpose	Key Functions
AnonymousVotingSystem	Core election logic, no admin, 2-of-3 EIP-712	`registerVoter()`, `vote()`, `getResults()`
MerkleTreeWithHistory	Incremental MiMC Merkle tree, depth 20	`isKnownRoot()`, `getLastRoot()`
Verifier	Groth16 proof verification	`verifyProof()`

```
npx hardhat compile    # compile all contracts
npx hardhat test       # run the full test suite
```

8.1 AnonymousVotingSystem

- Immutable configuration: `approvalThreshold`, `registrationStart/End`, `votingStart/End` and the verifier address; the constructor requires at least 2 candidates, a threshold of exactly 2, and valid, ordered time windows.
- `getCurrentPhase` — derives the phase from `block.timestamp`.
- `getDomainSeparator`, `getDigest`, `recoverSigner` — EIP-712 hashing and `ecrecover`-based signature recovery.
- `registerVoter(commitment, identityHash, signatures[2], validators[2])` — requires the Registration phase, an unused identity hash and commitment, two distinct registered validators and two valid signatures; then inserts the commitment into the Merkle tree.
- `vote(candidateId, nullifier, root, proof_a, proof_b, proof_c)` — requires the Voting phase, a valid candidate ID (1 to n), an unused nullifier, a known root and a valid Groth16 proof; then records the nullifier and increments the tally.
- `getResults`, `getCandidates`, `getCandidateCount`, `getElectionConfig` — public read functions used by the UI and audit.
- Events: `Commit`, `VoterRegistered`, `VoteCast`.

8.2 MerkleTreeWithHistory

- Incremental binary Merkle tree, depth 20 (capacity $2^{20} = 1{,}048{,}576$ voters).
- Hashing through the on-chain MiMC-Sponge contract.
- A 30-root circular history buffer (`ROOT_HISTORY_SIZE = 30`) so proofs built against a slightly older root still verify during concurrent voting.

8.3 Verifier

- Generated Groth16 verifier; checks the pairing equation through the AltBn128 precompile at `0x08`.
- Public inputs: `[nullifierHash, root]`; constant proof size of about 128 bytes.

9 Relayer REST API

Implemented in `relayer/server.js` (Express, port 3001).

Method	Route	Purpose
GET	`/api/status`	Health, relayer address and balance, contract address, queue length, metrics.
GET	`/api/requests`	List pending registration requests for validators.
POST	`/api/requests`	Create a request from `voterIdentifier`, `commitment`, `identityHash`.
POST	`/api/requests/:id/sign`	Validator submits `validatorAddress` and `signature`; returns approval count and `isReadyToRelay`.
POST	`/api/requests/:id/demo-approve`	Local-demo shortcut that signs as validator 0, 1 or 2 with Hardhat dev keys.
POST	`/api/relay/register`	Relays `registerVoter` (needs at least 2 signatures and addresses).
POST	`/api/relay/vote`	Relays `vote` from `candidateId`, `nullifier`, `root`, `proof_a`, `proof_b`, `proof_c`.
GET	`/api/audit`	Public audit: commitments, votes, duplicate-nullifier check, PASSED/FAILED.
GET	`/api/metrics`	Totals, average gas per vote, recent transactions, queue status.
POST	`/api/advance-phase`	Fast-forwards Hardhat time to the next phase (demos only).

Transaction queue. Every on-chain call goes through a `TransactionQueue`: jobs are processed one at a time, the nonce is fetched once and then incremented locally, and a failed job resets the nonce and is retried up to 3 times. This removes nonce collisions and transaction loss.

Example vote request body:

```
{
  "candidateId": 1,
  "nullifier": "0x...",
  "root": "0x...",
  "proof_a": ["0x..", "0x.."],
  "proof_b": [["0x..","0x.."],["0x..","0x.."]],
  "proof_c": ["0x..", "0x.."]
}
```

10 Current Local Deployment

From `deployed-contracts.json` (local Hardhat node):

Item	Value
Network (Chain ID)	localhost / Hardhat (31337)
AnonymousVotingSystem	`0x2279B7A0a67DB372996a5FaB50D91eAA73d2eBe6`
MiMCSponge	`0x0165878A594ca255338adfa4d48449f69242Eb8F`
Verifier	`0xa513E6E4b8f2a923D98304ec87F64353C4D5C853`

Addresses are regenerated on every fresh `npx hardhat node` restart.

10.1 Roles and Accounts in the Local Demo

Account	Role
Account 0 (`0xf39F...2266`)	Deployer; no administrative power after deployment
Account 1 (`0x7099...79C8`)	Validator 1
Account 2 (`0x3C44...93BC`)	Validator 2
Account 3 (`0x90F7...b906`)	Validator 3
Account 4 (`0x15d3...6A65`)	Relayer; pays gas for registrations and votes

11 Current Limitations

- **Validator collusion:** eligibility is checked off-chain; if 2 of 3 validators collude they can approve fictitious voters.
- **Validator linkability:** validators see the pair (identityHash, commitment) at registration, so a voter who reveals their secret note can be traced.
- **Relayer metadata and censorship:** the relayer does not log IPs, but network-level observers could correlate timing; a malicious relayer could drop votes. Direct MetaMask submission is a fallback but exposes the voter's wallet.
- **No coercion resistance or receipt-freeness:** a briber can demand the secret note and check the public `VoteCast` event.
- **Trusted setup:** Groth16 needs a circuit-specific ceremony; compromised "toxic waste" would allow forged proofs.
- **Immutable configuration:** a wrong candidate, address or schedule cannot be patched; the contract must be redeployed.
- **Consensus assumption:** finality depends on the underlying EVM network's security.
- **Local testbed:** benchmarks ran on a Hardhat node; public networks add dynamic fees and block gas limits, so Layer 2 would be needed at scale.
- **Demo-only shortcuts:** `/api/advance-phase` and `/api/demo-approve` rely on Hardhat features and public development keys and must not be exposed in a real deployment.

12 Sancus Comparison and Benchmark Results

This section records how the project resolves the ten limitations of the reference paper, with measured data from `benchmark-results.json`.

12.1 Sancus Limitations vs. This System

Ref	Sancus limitation	Our approach
L1	Niche BigchainDB platform	Native EVM with Solidity 0.8.20
L2	Eligibility check unimplemented	2-of-3 EIP-712 signatures, Merkle root check, Groth16 proof, nullifier registry
L3	Only 3–9 nodes evaluated	Benchmarks at 10, 50, 100 and 250 voters
L4	One transaction per block	Relayer queue; several transactions per mined block
L5	Transaction loss under parallel load	FIFO queue, sequential nonces: 105 concurrent votes, 0 lost
L6	Three central authorities must not collude	No token or identity-mapping authorities; privacy from ZK math; no admin
L7	Tor or proxies needed for anonymity	Relayer pays gas; voter wallet never on-chain; zero IP logging
L8	Complex voter experience	Simple browser wizard; no command-line tools for voters
L9	Only throughput and latency measured	Gas, proof time, latency and throughput all measured
L10	Relied on trusted inspectors	Public zero-trust audit page and `/api/audit`

12.2 Measured Benchmarks

Voters	Reg. time	Vote time	Throughput	Proof time	Gas / vote	Loss
10	0.97 s	3.36 s	2.98 tx/s	3,361 ms	348,220	0%
50	4.61 s	17.38 s	2.88 tx/s	3,476 ms	347,890	0%
105 (flood)	6.85 s	4.82 s	21.79 tx/s	3,410 ms	348,100	0%

- Browser WebAssembly generates a full Groth16 proof in about 3.4 seconds.
- On-chain verification costs a constant ≈348,000 gas per vote, independent of the number of voters.
- Registration costs about 1.06–1.08 million gas per voter because the Merkle tree insertion runs 20 MiMC-Sponge hashes on-chain.
- The relayer processed 105 simultaneous votes in 4.82 seconds with 100% confirmation.

13 Troubleshooting

Symptom / revert message	Fix
"Registration is not active"	Election is not in the Registration phase; use *Next Phase (Demo)* or wait for the schedule.
"Voting is not active"	Voting window not open yet or already closed.
"Identity already registered"	That student ID (identityHash) has already registered; each identity can register once.
"Commitment already registered"	The same commitment was submitted twice; generate a new secret note.
"First/Second signer is not a registered validator"	Signature came from an address not in `election.config.json`; sign with accounts 1–3.
"Invalid signature from first/second validator"	Signature does not match the commitment and identityHash; re-sign the request.
"Nullifier already used - double voting prohibited"	This voter has already voted; this is expected behaviour.
"Merkle root is invalid or expired"	Proof used a root older than the 30-root buffer or unregistered; rebuild the tree and prove again.
"Invalid candidate ID"	Candidate IDs run from 1 to the number of candidates.
Relayer returns 400	A required field is missing in the request body.
Relayer cannot find contract	`deployed-contracts.json` is missing; run `npm run deploy` first, then restart the relayer.
MetaMask wrong network or nonce errors	Switch to localhost (Chain ID 31337); after restarting the Hardhat node, reset the account in MetaMask.
Phase does not advance	`/api/advance-phase` needs the local Hardhat node running.
Proof generation is slow or fails	Use a recent browser; make sure `verifier.zkey` (14.9 MB) is present in `frontend/public`.

14 Repository Map

```
AnonymousVotingSystem/
  circuits/         Verifier.circom, CommitmentHasher.circom,
                    MerkleTreeChecker.circom, Verifier.js, verifier.zkey
  contracts/        AnonymousVotingSystem.sol, MerkleTreeWithHistory.sol,
                    Verifier.sol
  frontend/
    src/components/ VoterView, ValidatorView, ResultsView,
                    AuditView, PerformanceView, PhaseBanner
    src/contracts/  Verifier.js, deployed-contracts.json,
                    zk-browser-helper.js
    src/            App.jsx, main.jsx, index.css
    public/         static ZK artifacts (verifier.zkey)
  relayer/          server.js (FIFO queue, REST API)
  scripts/          deploy.js, zk-helper.js, simulate.js, stress-test.js
  test/             AnonymousVotingSystem.test.js (13 tests)
  election.config.json, deployed-contracts.json,
  benchmark-results.json, hardhat.config.js, package.json
  BENCHMARK.md, LIMITATIONS.md, HOW_TO_TEST.txt,
  VIVA_DEMO_SCRIPT.md, REPORT_SECTION.md, README.md
```

Source: `https://github.com/AtharvadChaudhari05/AnonymousVotingSystem`